

CO3-AT1

ASSESSMENT TOOL 1

1, Design and conduct an experiment to implement a Naïve Bayes classifier on a real-world dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.

INPUT

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, precision_score, recall_score

# Load real-world Iris dataset
iris = load_iris()
X, y = iris.data, iris.target

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

# Different feature sets
features = {
    "All Features": [0,1,2,3],
    "Sepal Features": [0,1],
    "Petal Features": [2,3]
}

print("NAIVE BAYES CLASSIFIER")
print("-" * 65)

for name, f in features.items():

    # Train Naive Bayes
    model = GaussianNB()
    model.fit(X_train[:, f], y_train)

    # Prediction
    pred = model.predict(X_test[:, f])

    # Evaluation
    acc = accuracy_score(y_test, pred)
    precision = precision_score(y_test, pred, average="macro")
    recall = recall_score(y_test, pred, average="macro")
```

OUTPUT

NAIVE BAYES CLASSIFIER

Feature Set: All Features

Accuracy : 91.11 %

Precision: 0.916

Recall : 0.911

Feature Set: Sepal Features

Accuracy : 68.89 %

Precision: 0.679

Recall : 0.689

Feature Set: Petal Features

Accuracy : 93.33 %

Precision: 0.935

Recall : 0.933

LAPLACE SMOOTHING COMPARISON

Smoothing = 1e-09 Accuracy = 91.11 %

Smoothing = 0.01 Accuracy = 91.11 %

Smoothing = 0.1 Accuracy = 88.89 %

2. Perform an experiment to estimate parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation approach. Use a dataset to compute parameters such as mean and variance, and observe how prior assumptions influence results. Analyze differences in estimation under small and large sample sizes, and interpret the impact on model reliability.

INPUT

```
import numpy as np

# Dataset: electricity consumption (kWh)
data = np.array([18, 20, 21, 19, 22, 24, 23, 21, 20, 25,
                  19, 22, 24, 23, 21, 26, 25, 22, 24, 23])

# Bayesian prior
prior_mean = 21
prior_var = 4

def estimate(data):
    n = len(data)

    # MLE
    mle_mean = np.mean(data)
    mle_var = np.var(data)      # MLE variance

    # Bayesian mean estimation
    bayes_mean = ((n / mle_var) * mle_mean +
                  (1 / prior_var) * prior_mean) / \
                  ((n / mle_var) + (1 / prior_var))

    # Bayesian variance (prior + observed variance)
    bayes_var = ((n * mle_var) + prior_var) / (n + 1)

    return mle_mean, mle_var, bayes_mean, bayes_var

# Small and large samples
samples = {
    "Small Sample (5)": data[:5],
    "Large Sample (20)": data
}

print("MLE vs BAYESIAN ESTIMATION")
print("-" * 65)
```

OUTPUT

MLE vs BAYESIAN ESTIMATION

```
-----  
  
Small Sample (5)  
Data: [18 20 21 19 22]  
MLE Mean      : 20.0  
MLE Variance  : 2.0  
Bayesian Mean : 20.091  
Bayesian Var  : 2.333  
  
Large Sample (20)  
Data: [18 20 21 19 22 24 23 21 20 25 19 22 24 23 21 26 25 22 24 23]  
MLE Mean      : 22.1  
MLE Variance  : 4.69  
Bayesian Mean : 22.039  
Bayesian Var  : 4.657  
  
Effect of Different Prior Means  
-----  
Prior Mean: 15 -> Bayesian Mean: 19.545  
Prior Mean: 21 -> Bayesian Mean: 20.091  
Prior Mean: 30 -> Bayesian Mean: 20.909
```